

**Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет прикладної математики
Кафедра системного програмування і спеціалізованих
комп'ютерних систем**

ЛАБОРАТОРНА РОБОТА № 1

*з дисципліни
«Бази даних та засоби управління»*

**ТЕМА: “ Проектування бази даних та ознайомлення з базовими
операціями СУБД PostgreSQL”**

**Група: КВ-34
Виконав: Протченко П.О
Оцінка:**

Зміст

Зміст	2
1.Вступ Варіант	3
2.Порядок виконання роботи.....	5
3.Виконання.....	6
1. Розробити модель «сутність-зв'язок» предметної галузі, обраної студентом самостійно, відповідно до пункту «Вимоги до ER-моделі».	6
2. Перетворити розроблену модель у схему бази даних (таблиці) PostgreSQL.	8
3. Виконати нормалізацію схеми бази даних до третьої нормальної форми (3НФ).....	12
4. Ознайомитись із інструментарієм PostgreSQL та pgAdmin 4 та внести декілька рядків даних у кожен з таблиць засобами pgAdmin 4.....	14
4.1 Назви, типи та обмеження на стовпці	14
4.2. Валідні тестові дані	17
4.3. Негативні перевірки	22
4.Висновок:.....	25

1.Вступ

Варіант

Система обліку екзаменаційних балів студентів

Посилання на GitHub

[GitHub](#)

Телеграм

@PR0TX

Теоретичні відомості

ER-модель — спосіб концептуального опису предметної галузі у вигляді **сутностей** (entities), **атрибутів** (attributes) і **зв'язків** (relationships) з кардинальностями (1:1, 1:N, N:M).

Нотація Crow's Foot використовує «лапки/гілки» для позначення кратності: одна риска — «1», кружок — «0», «лапка» — «багато».

Типи зв'язків:

- **1:1** — кожному екземпляру лівої сутності відповідає не більше одного праворуч (рідко, часто зливається в одну таблицю).
- **1:N** — один батьківський об'єкт має багато дочірніх (реалізується FK у «багато»).
- **N:M** — реалізується **асоціативною** таблицею (містить принаймні два FK, часто — свої атрибути).

Ключі:

- **PK (Primary Key)** — унікальний ідентифікатор рядка (прості або складові).
- **FK (Foreign Key)** — посилання на PK іншої таблиці; забезпечує посилальну цілісність.
- **UNIQUE** — гарантує унікальність значень у стовпці/комбінації стовпців.
- **NOT NULL** — забороняє порожні значення.
- **CHECK** — логічні обмеження (діапазони, маски).
- **ON DELETE / ON UPDATE** — дії при видаленні/оновленні батьківського рядка (CASCADE, RESTRICT, SET NULL, SET DEFAULT).
-

Нормалізація (1НФ→2НФ→3НФ):

- **1НФ**: атомарність значень, відсутність повторюваних груп.
- **2НФ**: немає часткових залежностей від частини **складового** ключа.
- **3НФ**: немає **транзитивних** залежностей неключових атрибутів; кожний неключовий залежить **тільки** від ключа, від цілого ключа і від нічого, крім ключа.
- Мета — усунути **аномалії** вставки/оновлення/видалення, мінімізувати дублювання, зробити схему логічно чистою.

2.Порядок виконання роботи

1. Розробити модель «сутність-зв'язок» предметної галузі, обраної студентом самостійно, відповідно до пункту «Вимоги до ER-моделі».
2. Перетворити розроблену модель у схему бази даних (таблиці) PostgreSQL.
3. Виконати нормалізацію схеми бази даних до третьої нормальної форми (3НФ).
4. Ознайомитись із інструментарієм PostgreSQL та pgAdmin 4 та внести декілька рядків даних у кожен з таблиць засобами pgAdmin 4.

3.Виконання

1. Розробити модель «сутність-зв’язок» предметної галузі, обраної студентом самостійно, відповідно до пункту «Вимоги до ER-моделі».

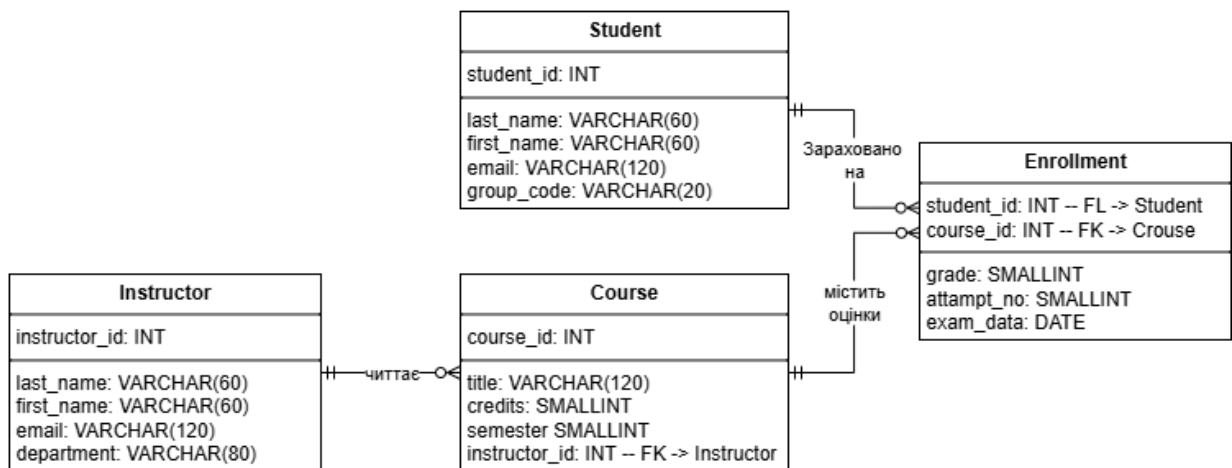
Система обліку екзаменаційних балів студентів.

Студент(Ім'я, прізвище, пошта, група

Викладач(Ім'я, прізвище, пошта, кафедра

Курс(Назва, кредити(ECTS), семестр, кафедра, викладач

Зарахування(Студент, курс, оцінка, спроба, дата)



Назва нотації

Для побудови ER-діаграми використано нотацію “Пташиної лапки (Crow’s Foot)”

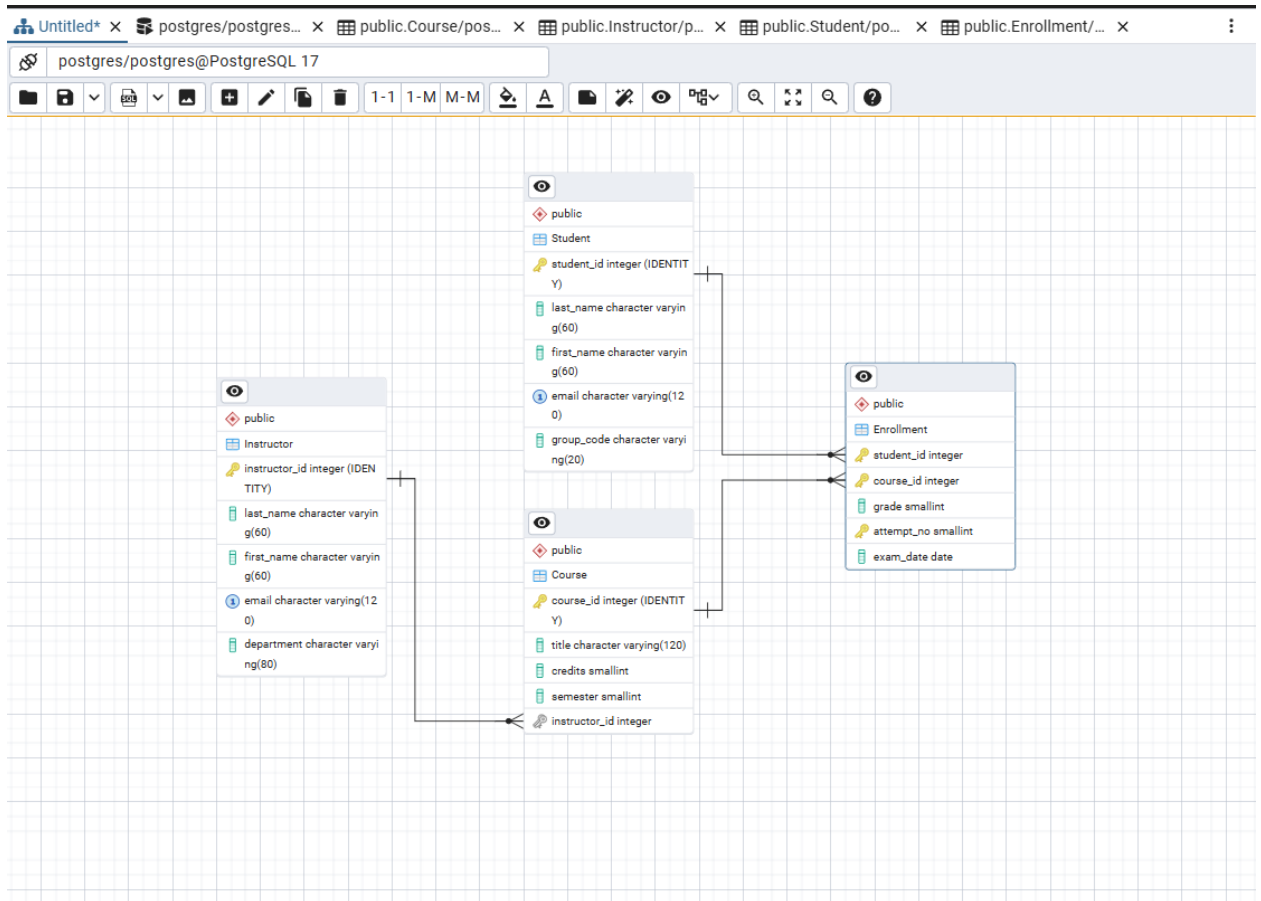
Перелік сутностей та призначення

1. **Student (Студент)** — зберігає довідкові дані про студентів: ПІ, email, навчальну групу.
Ключ: student_id.
2. **Instructor (Викладач)** — зберігає довідкові дані про викладачів: ПІ, email, кафедру/департамент.
Ключ: instructor_id.
3. **Course (Курс)** — описує навчальні дисципліни: назва, кредити, семестр, а також викладача, який читає курс.
Ключ: course_id; зовнішній ключ: instructor_id → Instructor.
4. **Enrollment (Зарахування)** — асоціативна сутність для реєстрації студентів на курси та фіксації результатів оцінювання. Містить атрибути зв’язку: оцінка, номер спроби, дата іспиту.
Складовий ключ: (student_id, course_id, attempt_no); зовнішні ключі: student_id → Student, course_id → Course.

Опис зв’язків і кардинальностей

- **Instructor 1:N Course** — один викладач може читати нуль або багато курсів; кожен курс має рівно одного викладача.
- **Student N:M Course** — студент може бути зарахований на багато курсів, а курс може мати багато студентів; зв’язок реалізовано через **Enrollment**, де додатково зберігаються атрибути оцінювання (grade, attempt_no, exam_date).

2. Перетворити розроблену модель у схему бази даних (таблиці) PostgreSQL.



Також після створення було вручну додано CHECK

The screenshot shows the 'Check' constraints for two tables in the PostgreSQL GUI:

Course

Name	Check
chk_course_credits_range	credits >= 1 AND credits <= 10
chk_course_semester_range	semester >= 1 AND semester <= 12

Enrollment

Name	Check
chk_enroll_grade_range	grade >= 0 AND grade <= 100
chk_enroll_attempt_nonneg	attempt_no >= 1

1:N (Instructor → Course)

Зв'язок реалізовано зовнішнім ключем `Course.instructor_id →`

`Instructor.instructor_id` з дією `ON DELETE RESTRICT` (забороняє видалити викладача, якщо є його курси), `ON UPDATE CASCADE`.

M:N (Student ↔ Course) з атрибутами оцінювання.

Зв'язок розкладено через асоціативну таблицю `Enrollment`, що містить атрибути зв'язку: `grade`, `attempt_no`, `exam_date`.

Оскільки для одного студента на одному курсі може бути кілька спроб, первинний ключ обрано складовий:

Primary Key (`student_id`, `course_id`, `attempt_no`).

Зовнішні ключі:

`Enrollment.student_id → Student.student_id`,

`Enrollment.course_id → Course.course_id`.

1. Унікальність та валідація довідкових даних.

Для `Student.email` і `Instructor.email` запроваджено `UNIQUE`.

2. Обмеження цілості та валідності.

Для атрибутів задано перевірки: `grade` у діапазоні `0..100`, `attempt_no` ≥ 1 ; `credits` та `semester`.

Реляційна схема

Таблиця Instructor

- **PK:** instructor_id (IDENTITY).
- **Атрибути:**
last_name VARCHAR(60) NOT NULL,
first_name VARCHAR(60) NOT NULL,
email VARCHAR(120) NOT NULL **UNIQUE**,
department VARCHAR(80) NOT NULL.

Таблиця Student

- **PK:** student_id (IDENTITY).
- **Атрибути:**
last_name VARCHAR(60) NOT NULL,
first_name VARCHAR(60) NOT NULL,
email VARCHAR(120) NOT NULL **UNIQUE**,
group_code VARCHAR(20) NOT NULL.

Таблиця Course

- **PK:** course_id (IDENTITY).
- **Атрибути:**
title VARCHAR(120) NOT NULL,
credits SMALLINT NOT NULL **CHECK 1..10**,
semester SMALLINT NOT NULL **CHECK 1..12**,
instructor_id INT NOT NULL **FK** → **Instructor(instructor_id)**
(ON UPDATE CASCADE, ON DELETE RESTRICT).

Таблиця Enrollment (асоціативна)

- **PK (складовий):** (student_id, course_id, attempt_no).
- **FK:**
student_id → **Student(student_id)** (ON UPDATE CASCADE, ON DELETE CASCADE),
course_id → **Course(course_id)** (ON UPDATE CASCADE, ON DELETE CASCADE).
- **Атрибути:**
attempt_no SMALLINT NOT NULL **CHECK ≥ 1**,
grade SMALLINT NOT NULL **CHECK 0..100**,
exam_date DATE NOT NULL.

Код SQL

BEGIN;

CREATE TABLE IF NOT EXISTS public."Instructor"

```
(
    instructor_id integer NOT NULL GENERATED ALWAYS AS IDENTITY,
    last_name character varying(60) NOT NULL,
    first_name character varying(60) NOT NULL,
    email character varying(120) NOT NULL,
    department character varying(80) NOT NULL,
    PRIMARY KEY (instructor_id),
    CONSTRAINT uq_instructor_email UNIQUE (email)
);
```

CREATE TABLE IF NOT EXISTS public."Student"

```
(
    student_id integer NOT NULL GENERATED ALWAYS AS IDENTITY,
    last_name character varying(60) NOT NULL,
    first_name character varying(60) NOT NULL,
    email character varying(120) NOT NULL,
    group_code character varying(20) NOT NULL,
    PRIMARY KEY (student_id),
    CONSTRAINT uq_student_email UNIQUE (email)
);
```

CREATE TABLE IF NOT EXISTS public."Course"

```
(
    course_id integer NOT NULL GENERATED ALWAYS AS IDENTITY,
    title character varying(120) NOT NULL,
    credits smallint NOT NULL,
    semester smallint NOT NULL,
    instructor_id integer NOT NULL,
    PRIMARY KEY (course_id)
);
```

CREATE TABLE IF NOT EXISTS public."Enrollment"

```
(
    student_id integer NOT NULL,
    course_id integer NOT NULL,
    grade smallint NOT NULL,
    attempt_no smallint NOT NULL,
    exam_date date NOT NULL,
    PRIMARY KEY (student_id, course_id, attempt_no)
);
```

ALTER TABLE IF EXISTS public."Course"

```
ADD CONSTRAINT fk_course_instructor FOREIGN KEY (instructor_id)
REFERENCES public."Instructor" (instructor_id) MATCH SIMPLE
ON UPDATE CASCADE
ON DELETE RESTRICT;
```

ALTER TABLE IF EXISTS public."Enrollment"

```
ADD CONSTRAINT fk_enroll_student FOREIGN KEY (student_id)
REFERENCES public."Student" (student_id) MATCH SIMPLE
ON UPDATE CASCADE
ON DELETE CASCADE;
```

ALTER TABLE IF EXISTS public."Enrollment"

```
ADD CONSTRAINT fk_enroll_course FOREIGN KEY (course_id)
REFERENCES public."Course" (course_id) MATCH SIMPLE
ON UPDATE CASCADE
ON DELETE CASCADE;
```

END;

3. Виконати нормалізацію схеми бази даних до третьої нормальної форми (3НФ).

1НФ: у кожній клітинці — одна, атомарна величина; жодних повторюваних груп/наборів у стовпцях.

2НФ: таблиця в 1НФ і немає часткових залежностей — жоден неключовий атрибут не залежить від частини складового ключа.

3НФ: таблиця в 2НФ і немає транзитивних залежностей неключових атрибутів (формально: для кожної нетривіальної ФЗ $X \rightarrow A$ виконується, що X — суперключ або A — «праймівий» атрибут).

Instructor(instructor_id PK, last_name, first_name, email UNIQUE, department)

FD:

- instructor_id $\rightarrow$ last_name, first_name, email, department
- Завдяки UNIQUE, також: email $\rightarrow$ instructor_id (email — альтернативний ключ).

Висновок: 1НФ — атрибути атомарні; 2НФ — ключ простий; 3НФ — транзитивних залежностей немає (атрибути описують тільки викладача; жоден неключовий не визначає інший неключовий)

Student(student_id PK, last_name, first_name, email UNIQUE, group_code)

FD:

- student_id $\rightarrow$ last_name, first_name, email, group_code
- email $\rightarrow$ student_id (альтернативний ключ).

Висновок: 1НФ — атомарно; 2НФ — ключ простий; 3НФ — транзитивних залежностей немає.

Course(course_id PK, title, credits, semester, instructor_id FK)

FD:

- course_id $\rightarrow$ title, credits, semester, instructor_id
- **Висновок:** 1НФ — атомарно; 2НФ — ключ простий; 3НФ — немає неключового, що визначає інший неключовий усередині цієї ж таблиці.

Enrollment(student_id FK, course_id FK, attempt_no, grade, exam_date, PK
= {student_id, course_id, attempt_no})

FD (кілька спроб на курс):

- $\langle \text{student_id}, \text{course_id}, \text{attempt_no} \rangle \rightarrow \text{grade}, \text{exam_date}.$

Висновок: **1НФ** — атрибути атомарні. **2НФ** — немає часткової залежності: і grade, і exam_date залежать від **усього** складового ключа, включно з attempt_no (без attempt_no ідентифікації конкретної спроби не буде). **3НФ** — транзитивних залежностей між неключовими (наприклад, grade не визначає exam_date за означенням) немає.

4. Ознайомитись із інструментарієм PostgreSQL та pgAdmin 4 та внести декілька рядків даних у кожну з таблиць засобами pgAdmin 4.

4.1 Назви, типи та обмеження на стовпці

Таблиця Instructor

Instructor

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Inherited from table(s)Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	instructor_id	integer			☒	☒	

GeneralDefinitionConstraintsVariablesSecurity

Default

Not NULL?☒

Type

NONEIDENTITYGENERATED

IdentityBY DEFAULT

	last_name	character varying	60		☒	☐	
	first_name	character varying	60		☒	☐	
	email	character varying	120		☒	☐	
	department	character varying	80		☒	☐	

CloseResetSave

Instructor

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary KeyForeign KeyCheckUniqueExclude

Name	Columns
Instructor_pkey	instructor_id

Instructor

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary KeyForeign KeyCheckUniqueExclude

Name	Columns
uq_instructor_email	email

Таблица Student

Student

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Inherited from table(s)

Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
 	student_id	integer			☒	☒	

GeneralDefinitionConstraintsVariablesSecurity

Default

Not NULL?☒

Type

NONEIDENTITYGENERATED

Identity

BY DEFAULT

 	last_name	character varying	60		☒	☐	
 	first_name	character varying	60		☒	☐	
 	email	character varying	120		☒	☐	
 	group_code	character varying	20		☒	☐	



Close

Reset

Save

Student

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary KeyForeign KeyCheckUniqueExclude

Name	Columns
Student_pkey	student_id

Student

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary KeyForeign KeyCheckUniqueExclude

Name	Columns
uq_student_email	email

Таблица Course

Course

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Inherited from table(s)

Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	course_id	integer			☒	☒	

General

Definition

Constraints

Variables

Security

Default

Not NULL?

Type

Identity

NONE

IDENTITY

GENERATED

BY DEFAULT

	title	character varying	120		☒	☐	
	credits	smallint			☒	☐	
	semester	smallint			☒	☐	
	instructor_id	integer			☒	☐	

Close

Reset

Save

Course

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary Key

Foreign Key

Check

Unique

Exclude

	Name	Columns
	Course_pkey	course_id

Course

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary Key

Foreign Key

Check

Unique

Exclude

	Name	Columns	Referenced Table
	fk_course_instructor	(instructor_id) -> (instructor_id)	public.Instructor

Course

GeneralColumnsAdvancedConstraintsPartitionsParametersSecuritySQL

Primary Key

Foreign Key

Check

Unique

Exclude

	Name	Check
	chk_course_credits_range	credits >= 1 AND credits <= 10
	chk_course_semester_range	semester >= 1 AND semester <= 12

4.2. Валідні тестові дані

Таблица Instructor

last_name	first_name	email	department
Коваленко	Олена	o.kovalenko@uni.edu	ІТ
Іванов	Петро	p.ivanov@uni.edu	Математика
Савчук	Дарія	d.savchuk@uni.edu	Фізика

stgres/postgres... x public.Course/pos... x public.Enrollment/... x public.Instructor/postgres/postgres@PostgreSQL 17 x public.Student/po... > v

public.Instructor/postgres/postgres@PostgreSQL 17

No limit

Query Query History Scratch Pad

```

1 SELECT * FROM public."Instructor"
2 ORDER BY instructor_id ASC

```

Data Output Messages Notifications

	Instructor_id [PK] integer	Last_name character varying (60)	First_name character varying (60)	Email character varying (120)	Department character varying (80)
0	3	Коваленко	Олена	o.kovalenko@uni.edu	ІТ
1	2	Іванов	Петро	p.ivanov@uni.edu	Математика
2	1	Савчук	Дарія	d.savchuk@uni.edu	Фізика

Total rows: 0 Query complete 00:00:00.043 CRLF Ln 1, Col 1

Таблица Enrollment

Enrollment

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Inherited from table(s)

Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	student_id	integer			☒	☒	
	course_id	integer			☒	☒	
	grade	smallint			☒	☐	
	attempt_no	smallint			☒	☒	1
	exam_date	date			☒	☐	CURRENT

Close

Reset

Save

Enrollment

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key

Foreign Key

Check

Unique

Exclude

Name	Columns
Enrollment_pkey	student_id,course_id,attempt_no

Enrollment

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key

Foreign Key

Check

Unique

Exclude

Name	Columns	Referenced Table
fk_enroll_course	(course_id) -> (course_id)	public.Course
fk_enroll_student	(student_id) -> (student_id)	public.Student

Enrollment

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key

Foreign Key

Check

Unique

Exclude

Name	Check
chk_enroll_grade_range	grade >= 0 AND grade <= 100
chk_enroll_attempt_nonneg	attempt_no >= 1

Таблиця Student

last_name	first_name	email	group_code
Сидоренко	Марія	m.sydoorenko@uni.edu	KB-21
Бондар	Андрій	a.bondar@uni.edu	KB-22
Ченко	Ілля	i.chenko@uni.edu	KB-23

stgres/postgres... x public.Course/pos... x public.Enrollment/... x public.Instructor/p... x public.Student/postgres/postgres@PostgreSQL 17 x

public.Student/postgres/postgres@PostgreSQL 17

The session is idle and there is no current transaction.

Query Query History Scratch Pad x

1 SELECT * FROM public."Student"
2 ORDER BY student_id ASC

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	student_id [PK] integer	last_name character varying (60)	first_name character varying (60)	email character varying (120)	group_code character varying (20)
1	2	Сидоренко	Марія	m.sydoorenko@uni.edu	KB-21
2	3	Бондар	Андрій	a.bondar@uni.edu	KB-22
3	4	Ченко	Ілля	i.chenko@uni.edu	KB-23

Total rows: 3 Query complete 00:00:00.035 CRLF Ln 1, Col 1

Таблица Course

title	credits	semester	instructor_id
Бази даних	5	3	1
Алгоритми	6	3	2
Вища математика	4	1	2

public.Course/postgres/postgres@PostgreSQL 17

No limit

Query History Scratch Pad x

```

1 SELECT * FROM public."Course"
2 ORDER BY course_id ASC

```

Data Output Messages Notifications

	course_id [PK] integer	title character varying (120)	credits smallint	semester smallint	instructor_id integer
0	3	Бази даних	5	3	3
1	2	Алгоритми	6	3	2
2	1	Вища математика	4	1	1

Total rows: 0 Query complete 00:01:15.330 CRLF Ln 1, Col 1

Таблица Enrollment

student_id	course_id	attempt_no	grade	exam_date
1	1	1	58	2025-01-20
1	1	2	82	2025-02-10
2	2	1	74	2025-01-22
3	3	1	91	2025-01-18

public.Enrollment/postgres/postgres@PostgreSQL 17

The session is idle and there is no current transaction.

Query Query History

Scratch Pad x

1 SELECT * FROM public."Enrollment"
2 ORDER BY student_id ASC, course_id ASC, attempt_no ASC

Data Output Messages Notifications

	student_id [PK] integer	course_id [PK] integer	grade smallint	attempt_no [PK] smallint	exam_date date
0	1	1	58	1	2025-01-20
1	1	1	82	2	2025-02-10
2	2	2	74	1	2025-01-22
3	3	3	91	1	2025-01-18

Total rows: 0 Query complete 00:00:00.048 CRLF Ln 1, Col 1

4.3. Негативні перевірки

UNIQUE (email) у Student/Instructor

Очікування: помилка порушення унікальності (duplicate key value violates unique constraint ...).

	instructor_id [PK] integer	first_name character varying (60)	email character varying (120)	department character varying (80)
0+	[default]	Марія	o.kovalenko@uni.edu	IT
1	3	Олена	o.kovalenko@uni.edu	IT
2	2	Петро	p.ivanov@uni.edu	Математика
3	1	Дарія	d.savchuk@uni.edu	Фізика

повторювані значення ключа порушують обмеження унікальності "uq_instructor_email"
Ключ (email)=(o.kovalenko@uni.edu) вже існує.

	student_id [PK] integer	last_name character varying (60)	first_name character varying (60)	email character varying (120)	group_code character varying (20)
1+	[default]	Сидоренко	Максим	m.sydoenko@uni.edu	KB-33
2	3	Сидоренко	Марія	m.sydoenko@uni.edu	KB-21
3	2	Бондар	Андрій	a.bondar@uni.edu	KB-22
4	1	Ченко	Ілля	i.chenko@uni.edu	KB-23

повторювані значення ключа порушують обмеження унікальності "uq_student_email"
Ключ (email)=(m.sydoenko@uni.edu) вже існує.

Total rows: 0 Query complete 00:00:00.043 Changes staged: Added: 1

Правила БД відпрацювали правильно

NOT NULL

Очікування: помилка, що стовпець не може бути NULL.

	course_id [PK] integer	title character varying (120)	credits smallint	semester smallint	instructor_id integer
0+	[default]	[null]	2	2	3
1	3	Бази даних	5	3	3
2	2	Алгоритми	6	3	2
3	1	Вища математика	4	1	1

null значення в стовпці "title" відношення "Course" порушує not-null обмеження
Помилковий рядок містить (4, null, 2, 2, 3).

Total rows: 0 Query complete 00:01:15.330 Changes staged: Added: 1

Правила БД відпрацювали правильно

CHECK (діапазони)

Очікування: помилка перевірки CHECK (grade BETWEEN 0 AND 100) або (attempt_no ≥ 1).

student_id	course_id	grade	attempt_no	exam_date
integer	[PK] integer	smallint	[PK] smallint	date
0+	1	1	120	3
1	1	1	58	1
2	1	1	82	2
3	2	2	74	1
4	3	3	91	1

новий рядок для відношення "Enrollment" порушує перевірку обмеження перевірки "chk_enroll_grade_range"
Помилковий рядок містить (1, 1, 120, 3, 2026-01-25).

Total rows: 0 Query complete 00:00:00.048 Changes staged: Added: 1 CRLF Ln 1, Col 1

Правила БД відпрацювали правильно

FK (цілісність посилань)

Очікування: помилка порушення зовнішнього ключа (insert/update on table ... violates foreign key constraint ...).

course_id	title	credits	semester	instructor_id
[PK] integer	character varying (120)	smallint	smallint	integer
0+	[default]	Виш мат	5	4
1	3	Бази даних	5	3
2	2	Алгоритми	6	3
3	1	Вища математика	4	1

insert або update в таблиці "Course" порушує обмеження зовнішнього ключа "fk_course_instructor"
Ключ (instructor_id)=(999) не присутній в таблиці "Instructor".

Total rows: 0 Query complete 00:01:15.330 Changes staged: Added: 1 CRLF Ln 1, Col 1

Правила БД відпрацювали правильно

ON DELETE CASCADE / RESTRICT

Очікування: RESTRICT — видалення заборонено

instructor_id	last_name	first_name	email	department
[PK] integer	character varying (60)	character varying (60)	character varying (120)	character varying (80)
0	3	Коваленко	Олена	о.kovalenko@uni.edu
1	2	Іванов	Петро	p.ivanov@uni.edu
2	1	Савчук	Дарія	d.savchuk@uni.edu

update або delete в таблиці "Instructor" порушує обмеження зовнішнього ключа "fk_course_instructor" таблиці "Course"
На ключ (instructor_id)=(3) все ще є посилання в таблиці "Course".

Total rows: 0 Query complete 00:00:00.043 Rows selected: 1 Changes staged: Deleted: 1 CRLF Ln 1, Col 1

Правила БД відпрацювали правильно

Очікування: записи в Enrollment для студента **видаляться автоматично**

	id integer	course_id [PK] integer	grade smallint	attempt_no [PK] smallint	exam_date date
1	1	1	58	1	2025-01-20
2	1	1	82	2	2025-02-10
3	2	2	74	1	2025-01-22

✓ Successfully run. Total query runtime: 35 msec. 3 rows affected. ✕

Total rows: 3 Query complete 00:00:00.035 CRLF Ln 1, Col 1

Правила БД відпрацювали правильно

4.Висновок:

У ході виконання лабораторної роботи було послідовно реалізовано всі етапи проєктування та створення реляційної бази даних у середовищі PostgreSQL.

На першому етапі побудовано ER-модель у нотації **Crow's Foot**, яка дозволила чітко відобразити основні сутності предметної галузі «Система обліку екзаменаційних балів студентів», їхні атрибути та зв'язки з кардинальностями.

Далі модель була перетворена у схему бази даних: створено чотири таблиці (Student, Instructor, Course, Enrollment) із визначенням первинних і зовнішніх ключів, обмежень NOT NULL, UNIQUE, CHECK та правил підтримання посилальної цілісності. Особливу увагу приділено правильній реалізації зв'язку **М:Н** між студентами та курсами через асоціативну таблицю Enrollment, яка зберігає додаткові атрибути оцінювання та підтримує можливість кількох спроб складання іспиту.

Виконано нормалізацію до третьої нормальної форми (3НФ): показано функціональні залежності для кожної таблиці, доведено відсутність часткових і транзитивних залежностей. Це гарантує відсутність аномалій вставки, оновлення та видалення, а також мінімізує дублювання даних.

На завершальному етапі роботу з інструментарієм PostgreSQL та pgAdmin 4 було закріплено практикою: у таблиці додано валідні тестові дані, а також проведено негативні перевірки роботи обмежень (UNIQUE, NOT NULL, CHECK, FK, ON DELETE CASCADE/RESTRICT). Усі перевірки показали правильність роботи накладених правил цілісності.